FUNDOSCOPY STUDY · AI 기초

# Vision Mamba, 쉽게 이해하기

ViT를 아는 사람을 위한 SSM · Vision Mamba

ViT: $O(L^2)$ → Mamba: $O(L)$

요약

# 한 줄 요약

Attention이 "모든 토큰을 매번 전부 조회"라면, Vision Mamba는 **입력이 조절하는 요약 메모리(상태)를 양방향으로 순차 갱신**해서 O(L)로 같은 전역 문맥을 얻는다.

**Attention**
모든 토큰 쌍 비교 · $O(L^2)$

**SSM / Mamba**
요약 상태를 순차 갱신 · O(L)

**Vision Mamba**
양방향 스캔 + 지역 Conv

> 이 덱은 순서대로: ViT → SSM → 이산화(Δ) → 숫자 예 → 컨볼루션 → 병렬 스캔 → 선택성 → 코드 → 블록 → 비용.

# 왜 바꾸려는가

이미지를 패치 토큰 `x₁ ⋯x_L` 로 만들고, attention으로 **모든 토큰 쌍**을 비교한다. L이 커지면 연산·메모리가 **L²**로 폭증.

$$\mathbf{Attention}(Q,K,V)=\mathbf{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V,\qquad \mathbf{cost}=O(L^2D)$$

| L (토큰) | 해상도 예 | Attention 행렬 원소 수 | SSM 상태 크기 |
|---|---|---|---|
| 197 | 224² → 16² 패치 | 38,809 | 3,152 |
| 784 | 448² → 28² 패치 | 614,656 | 12,544 |

> L이 2배 → attention 4배, SSM 2배. 고해상도일수록 격차 폭증.

# 상태공간모델(SSM)

$$h'(t) = A\,h(t) + B\,x(t), \qquad y(t) = C\,h(t) + D\,x(t)$$

| 기호 | 의미 |
|---|---|
| h | **은닉 상태 = 정보의 압축 메모리** |
| A | 상태 전이 |
| B | 입력 주입 |
| C | 출력 읽기 |
| D | 지름길(skip) |

### 직관

Attention = 모든 토큰을 저장 후 매번 전부 조회.
SSM = **하나의 요약 메모리를 순서대로 갱신**하며 읽는다 → O(L).

# Δ는 어디서 오나 (ZOH)

구간 전체에서 입력이 일정하다고 보는 영차홀드 해:

$$\bar{A} = e^{\Delta A}, \qquad \bar{B} = \left(\int_0^{\Delta} e^{\tau A} d\tau\right) B = A^{-1}\left(e^{\Delta A} - I\right) B$$

Mamba는 행렬 역산을 피하려 1차 근사 $\bar{B} \approx \Delta B$ 사용. A=−1, B=1 일 때:

| Δ | 정확한 $\bar{B}$ | 근사 $\bar{B} = \Delta B$ |
|---|---|---|
| 0.5 | 0.3935 | 0.5000 |
| 1.0 | 0.6321 | 1.0000 |
| 2.0 | 0.8647 | 2.0000 |

> `A = -exp(A_log)` → A<0 → `Ā = e^(ΔA) ∈ (0,1)` → 재귀가 **수축** → 기울기 안정.

# 숫자로 따라가기 — Δ가 만드는 차이

`A=-1, B=C=1, x=[1, 0, 2]`

Δ = 0.5 : $\bar{A}$ = 0.6065, $\bar{B}$ = 0.5

| t | x_t | h_t |
|---|---|---|
| 1 | 1 | 0.5000 |
| 2 | 0 | 0.3033 |
| 3 | 2 | 1.1839 |

Δ = 2.0 : $\bar{A}$ = 0.1353, $\bar{B}$ = 2.0

| t | x_t | h_t |
|---|---|---|
| 1 | 1 | 2.0000 |
| 2 | 0 | 0.2707 |
| 3 | 2 | 4.0366 |

같은 입력, t3이 **4.04 vs 1.18 (약 3.4배)** — Δ가 크면 현재 입력 강하게·과거 빠르게 잊음.

# 같은 식, 두 가지 계산

### 추론: 재귀(순차)

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t$$

앞에서 뒤로 하나씩 · O(L)

### 학습: 컨볼루션(병렬)

$$y = x * K$$

고정 커널이면 병렬 · O(L)

```
전개:  K = ( C B̄ ,  C Ā B̄ ,  C Ā²  B̄ ,  ... )   # 고정 커널(LTI)
```

> 고정 커널은 재사용 가능하지만 "입력에 따라 선택"은 불가 → 다음 절(선택성).

4. 수식 ②

# 재귀 → 컨볼루션 유도

$$h_t = \sum_{s=1}^{t} \bar{A}^{\,t-s} \bar{B}\, x_s, \qquad y_t = \sum_{k=0}^{t-1} \underbrace{\left(C \bar{A}^k \bar{B}\right)}_{K_k} x_{t-k} = (K * x)_t$$

Ā=0.6065, B̄=0.5, C=1, x=[1,0,2] 로 검산:

| 커널 | 값 | y_t 검산 | 결과 |
|---|---|---|---|
| $K_0 = C\,\bar{B}$ | 0.5000 | $y_1$ = 0.5×1 | 0.5000 |
| $K_1 = C\,\bar{A}\,\bar{B}$ | 0.3033 | $y_2$ = 0 + 0.3033 | 0.3033 |
| $K_2 = C\,\bar{A}^2\,\bar{B}$ | 0.1839 | $y_3$ = 1.0 + 0 + 0.1839 | 1.1839 |

> 앞 슬라이드의 순차 결과(0.5 / 0.3033 / 1.1839)와 정확히 일치.

# 선택성 (Selective SSM)

Δ,B,C를 **입력 x_t마다 다르게** 만든다 → "무엇을 기억/무시할지"를 입력이 결정.

$$\Delta_t = \mathrm{softplus}(\mathrm{Linear}_\Delta(x_t)), \quad B_t = \mathrm{Linear}_B(x_t), \quad C_t = \mathrm{Linear}_C(x_t)$$

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t + D x_t$$

**LTI(상수)**

커널 고정 → 합성곱 가능, **선택 불가**

**Selective(입력 의존)**

커널이 매번 달라짐 → 합성곱 불가 → **병렬 스캔**

5. 수식 ③

# 병렬 스캔 (associative scan)

$$h_t = a_t h_{t-1} + b_t, \quad a_t = \bar{A}_t,\ b_t = \bar{B}_t x_t$$

$$(a_2, b_2) \circ (a_1, b_1) = \big(a_2 a_1,\ a_2 b_1 + b_2\big) \quad (\text{결합법칙 성립})$$

수치 예: `a=[0.5,0.5,0.5]`, `b=[1,0,2]`

| t | prefix (A_t, B_t) | h_t = B_t |
|---|---|---|
| 1 | (0.5000, 1.0000) | 1.0000 |
| 2 | (0.2500, 0.5000) | 0.5000 |
| 3 | (0.1250, 2.2500) | 2.2500 |

> 결합법칙 → O(L) 작업을 **O(log L) 깊이**로 병렬화 (Blelloch / CPU는 `tf.scan`).

# PyTorch — 정답 오라클(순차 루프)

```
import torch, torch.nn.functional as F

def selective_scan(u, delta, A, B, C, D=None):
    # u, delta: (b, d, l)   A: (d, n)   B, C: (b, n, l)
    b, d, l = u.shape; n = A.shape[1]
    delta = F.softplus(delta)                 # Δ > 0
    dA   = torch.exp(delta[..., None] * A)    # exp(ΔA) -> (b,d,l,n)
    dB_u = (delta[..., None]
            * B.transpose(1, 2)[:, None]
            * u[..., None])
    h = u.new_zeros((b, d, n)); ys = []
    for t in range(l):
        h = dA[:, :, t] * h + dB_u[:, :, t]   # h_t = Ā h_{t-1} + B̄ x_t
        ys.append(torch.einsum("bdn,bn->bd", h, C[:, :, t]))
    y = torch.stack(ys, dim=2)
    return y + u * D.view(1, -1, 1) if D is not None else y
```

이 구현이 **정답 기준**. 다음 슬라이드의 병렬 버전과 수치 일치(허용오차 < 1e-5)를 확인.

# TensorFlow — `tf.scan` 병렬화

```
import tensorflow as tf

def selective_scan_tf(u, delta, A, B, C):
    # u, delta: (b, l, d)   A: (d, n)   B, C: (b, l, n)
    delta = tf.nn.softplus(delta)
    dA  = tf.exp(delta[..., None] * A)                # (b,l,d,n)
    dBu = delta[..., None] * B[:, :, None, :] * u[..., None]

    def step(h, xs):
        dA_t, dBu_t, C_t = xs
        h = dA_t * h + dBu_t
        y = tf.reduce_sum(h * C_t[:, None, :], axis=-1)
        return h, y

    h0 = tf.zeros([tf.shape(u)[0], tf.shape(u)[2], A.shape[1]])
    _, ys = tf.scan(step, (tf.transpose(dA, [1,0,2,3]),
                           tf.transpose(dBu,[1,0,2,3]),
                           tf.transpose(C,  [1,0,2])), initializer=h0)
    return tf.transpose(ys, [1,0,2])
```

# Mamba 하이퍼파라미터

`d_state = 16, d_conv = 4, expand = 2, dt_rank = ceil(d_model/16)`

| d_model | d_inner (expand=2) | dt_rank | d_state | d_conv |
|---|---|---|---|---|
| 128 | 256 | 8 | 16 | 4 |
| 256 | 512 | 16 | 16 | 4 |
| 768 | 1536 | 48 | 16 | 4 |

> **안저 앱 계획 연결** — 이 파라미터를 그대로 써서 CPU 친화적 preset(tiny/small)을 구성한다.

# Vision Mamba 블록 — ViT 블록과 1:1

attention 자리에 **양방향 SSM**, MLP 자리에 **gating**.




### Conv1d

ViT에 없음 → 지역성 부여, 순서 의존성 완화

### ⊙ gate (SiLU)

MLP 대체 — 출력 통과량을 입력이 결정

# 양방향 스캔 — 이미지는 순서가 없다

한 방향 스캔은 패치 순서에 민감 → 정방향 + 역방향을 합친다.

$$y = \mathrm{SSM}_{\mathrm{forward}}(x) + \mathrm{flip}\big(\mathrm{SSM}_{\mathrm{backward}}(\mathrm{flip}(x))\big)$$

정방향 SSM →

← 역방향 SSM

ViT는 순열 불변(방향 없음), SSM은 방향 의존 → 양방향이 필수.

8. 비용

# 비용 비교 — 실제 숫자

d_model=768, 토큰 L개, **대략적** MAC 기준.

| L | 해상도 | Attention ≈ $2L^2D$ | SSM scan ≈ 3·L·d_inner·N | 배수 |
|---|---|---|---|---|
| 197 | $224^2$ | 59.6M | 14.5M | 4.1× |
| 784 | $448^2$ | 944.1M | 57.8M | 16.3× |

> Vim 논문: 1248×1248 배치 추론에서 DeiT 대비 약 **2.8배 빠르고 GPU 메모리 약 86.8% 절감**.

9. 정리

# ViT ↔ Vision Mamba

| | ViT | Vision Mamba |
|---|---|---|
| 패치 임베딩 | 있음 | 동일 |
| 위치 임베딩 | 있음 | 동일 (+ Conv1d) |
| 전역 상호작용 | Self-Attention | 양방향 SSM 순차 스캔 |
| 복잡도 | $O(L^2)$ | **O(L)** |
| 방향성 | 없음(순열 불변) | 있음 → 양방향 |
| 선택성 | softmax 가중치 | 입력 의존 Δ,B,C |
| 학습 병렬화 | 행렬곱 | 병렬 스캔 |

> **주의**: SSM의 "전역"은 상태(N)로 압축 전달 → 정보 병목 존재. 매우 긴 문맥의 세부 기억은 attention이 유리할 수 있다.

10. 연결

# 안저 AI 프로젝트와 연결 · 참고

- SSM은 선형·커널 무관 → **순수 TF 연산으로 CPU 구현** 가능 → TFLite 배포 유리.
- 안저/의료영상은 **CNN + Mamba 하이브리드**가 강세: MedMamba, StageMamba, LEMUR.
- `app/PLAN-keras-tensorflow.md` 전략 A = Keras 커스텀 SSM 레이어.

참고: Vision Mamba (ICML 2024, arXiv:2401.09417) · MedMamba (arXiv:2403.03849) · VMamba · LEMUR.